

COMPARATIVE ANALYSIS OF THREE OPTIMIZATION METHODS

- Steepest Descent, Newton's Method, BFGS -

Farah Piralić

Every method searches for the minimum of a function using the exact same update rule:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

$\mathbf{x}_k$ current position

$\mathbf{d}_k$ search direction — how each method differs

α_k step size — how far to move along it

The only difference between the three methods is how $\mathbf{d}_k$ is picked — from gradient step (Steepest Descent), to the Hessian (Newton), to a running approximation of it (BFGS).

Order of Information Used

Steepest Descent

1st-order — knows which way is downhill

BFGS

Builds a Hessian guess as it steps

Newton

2nd-order — calculates Hessian, takes into account the curvature

How Each Method Picks Its Direction

Steepest Descent

$$d = -g$$

Moves straight down the current gradient. Simple and cheap per step calculation.

1st-order

Newton

$$H \cdot d = -g$$

Solves a linear system with the full Hessian each step. Fast convergence, but expensive per step.

2nd-order

BFGS

$$d = -B \cdot g$$

Keeps a running approximation B of the inverse Hessian, refined after every step from the gradient change.

Quasi-Newton

Why Newton gives shape and not just slope

The gradient g is a vector of first derivatives — it says which way is downhill. The Hessian H is the matrix of second derivatives — it says how the slope itself is changing.

$$H(x,y) = \begin{vmatrix} \partial^2 f / \partial x^2 & \partial^2 f / \partial x \partial y \\ \partial^2 f / \partial y \partial x & \partial^2 f / \partial y^2 \end{vmatrix}$$

Newton solves $H \cdot d = -g$ for d instead of just following $-g$. This “undoes” the stretching of the bowl, so the direction points straight at the minimum — not just downhill.

Curvature, in Plain Terms

The Hessian's eigenvalues are the curvatures along each principal direction of the bowl — how sharply it curves that way.

A big eigenvalue $\rightarrow$ a steep, narrow direction. A small eigenvalue $\rightarrow$ a shallow, wide direction.

Their ratio is the condition number $\kappa = \lambda_{\max} / \lambda_{\min}$ — exactly the κ used to stretch the test bowls ($\kappa=2, 100, 2000$).

Large κ = a long, narrow valley. Steepest Descent zig-zags across it; Newton's H^{-1} rescales the step so it heads straight for the bottom regardless of κ .

Learning the Hessian's Shape from Gradients Alone

BFGS never computes a real Hessian. Instead it keeps a matrix B , a guess of the inverse Hessian and sharpens it after every step using only the change in gradient.

One BFGS Iteration

$$d = -B \cdot g$$

compute the direction from the current guess B

$$\alpha \text{ chosen by line search}$$

exact line search or Armijo

$$s = \alpha d, \quad x \leftarrow x + s$$

take the step, s is how far x moved

$$y = g_{\text{new}} - g_{\text{old}}$$

measure how much the gradient changed

$$\text{check } y^T s > 0$$

curvature condition, only update if it helps

$$\text{update } B \text{ via the BFGS formula}$$

B gets a little more accurate

The Curvature Pair

$$s_k = x_{k+1} - x_k \quad \text{step taken}$$

$$y_k = g_{k+1} - g_k \quad \text{gradient change}$$

$$y_k^T s_k > 0 \quad \text{— the curvature condition that guards the update}$$

The BFGS Update Formula

$$B_{k+1} = (I - \rho_k s_k y_k^T) B_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

$$\text{with } \rho_k = 1 / (y_k^T s_k)$$

Finding the Best Step Along the Chosen Direction

Quadratic Functions: Closed Form

$$\alpha^* = -g^T d / d^T H d$$

For other, curvier functions there is no simple formula — the app searches for the best α numerically instead:

Bracket it — Double the step while f keeps decreasing. The moment it turns back up, the true minimum lies between the last two steps.

Golden-section search — Pick two interior points, discard the side with the higher value, and repeat — the bracket shrinks until it's tiny.

Good enough not the perfect step

The Sufficient-Decrease Condition

Instead of finding the exact minimum along d , Armijo just checks that a candidate step decreases f by “enough” relative to the slope:

$$f(x+\alpha d) \leq f(x) + c_1 \alpha g^T d$$

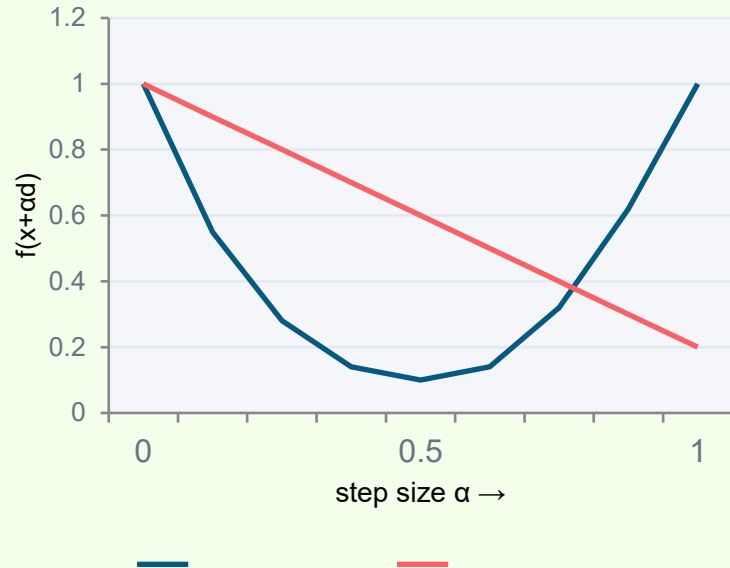
$f(x+\alpha d)$ actual function value after taking the step

$f(x) + c_1 \alpha g^T d$ the “acceptable” line — the tangent, scaled down by c_1

$c_1 = 10^{-4}$ a small safety constant from optimization theory

The Backtracking Loop

$\alpha \leftarrow 1$; while condition fails: $\alpha \leftarrow \alpha / 2$; take the first α that passes.



Any α where the actual curve dips below the Armijo bound is accepted — backtracking from $\alpha=1$ finds one fast.

Test Functions

All quadratic bowls share one general form; κ (condition number) and rotation angle θ control how stretched and tilted the bowl is:

$$f(x,y) = 0.5 (a_{11}x^2 + 2a_{12}xy + a_{22}y^2) \quad \text{where} \quad a_{11} = \cos^2\theta + \kappa\sin^2\theta, \quad a_{22} = \sin^2\theta + \kappa\cos^2\theta, \quad a_{12} = (1-\kappa)\cos\theta \cdot \sin\theta$$

Function	Parameters	Minimum at
Convex quadratic	$\kappa = 2, \quad \theta = 0^\circ$	$(0, 0)$
Ill-conditioned quadratic	$\kappa = 100, \quad \theta = 15^\circ$	$(0, 0)$
Rotated quadratic	$\kappa = 2000, \quad \theta = 30^\circ$	$(0, 0)$

Function**Exact Equation****Minimum at****Strongly convex (quad + exp)**

$$f(x,y) = x^2 + y^2 + \exp(0.1(x+y))$$

(-0.0495, -0.0495)

Double Well

$$f(x,y) = 0.25x^4 - 0.5x^2 + 0.5y^2$$

(-1,0) and (1,0); saddle at (0,0)

Rosenbrock

$$f(x,y) = (1-x)^2 + 100(y-x^2)^2$$

(1, 1)

Himmelblau

$$f(x,y) = (x^2+y-11)^2 + (x+y^2-7)^2$$

4 minima incl. (3,2), (-2.81,3.13)

What Happens When Curvature Information Misleads

Newton and BFGS both rely on curvature information that can be unreliable far from the minimum or near a saddle point

Newton Safeguard

Before stepping, check the Newton direction actually points downhill

If $H \cdot d = -g$ fails to solve, or the direction doesn't decrease f , fall back to plain Steepest Descent ($d = -g$) for that one step

BFGS Safeguard

Before updating B , check the curvature condition $y^T s > 0$

If an update would make the approximation worse, skip it and keep the old B

If B ever becomes numerically unstable, reset it (*reset B behaves just like Steepest Descent for that step.*)

The App

Control Panel

Choose method, test function, step-size rule,
starting point, tolerance...

Contour Plot

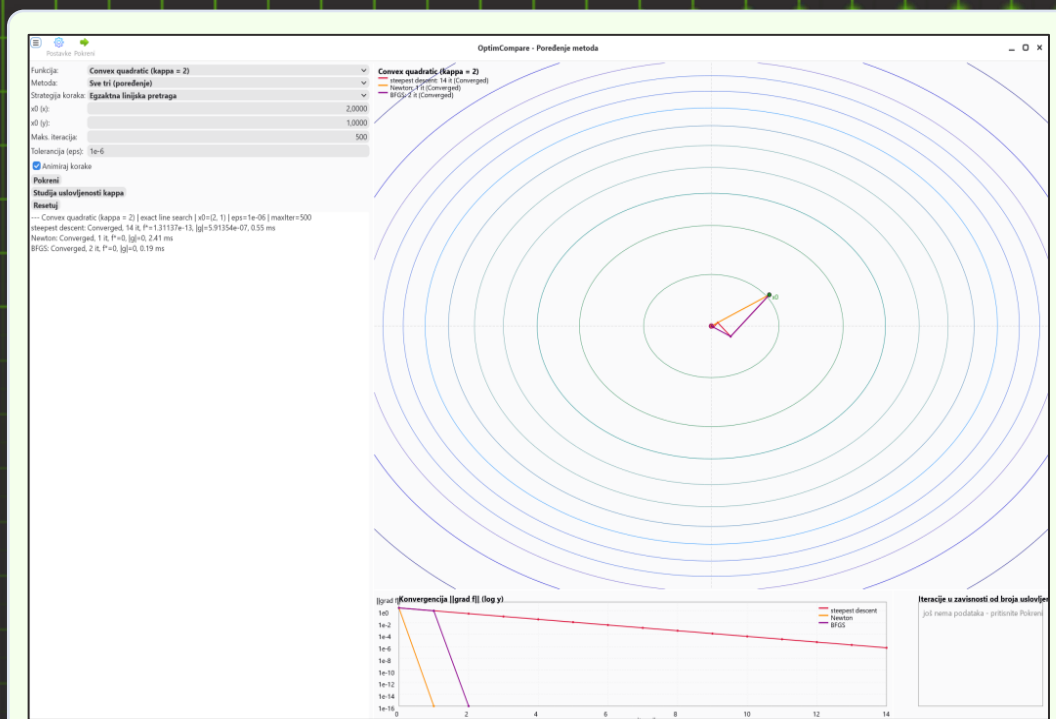
Level curves of the chosen function with each methods path drawn on top.

Convergence Chart

Gradient norm vs. iteration, log-scale — a fast drop means fast convergence.

Condition-Number Chart

Iterations needed as κ increases on a fixed stretched bowl



Convex quadratic ($\kappa = 2$) — control panel, contour plot, and convergence chart together

Stress-Testing Each Method as the Bowl Gets Longer and Thinner

What the Study Does

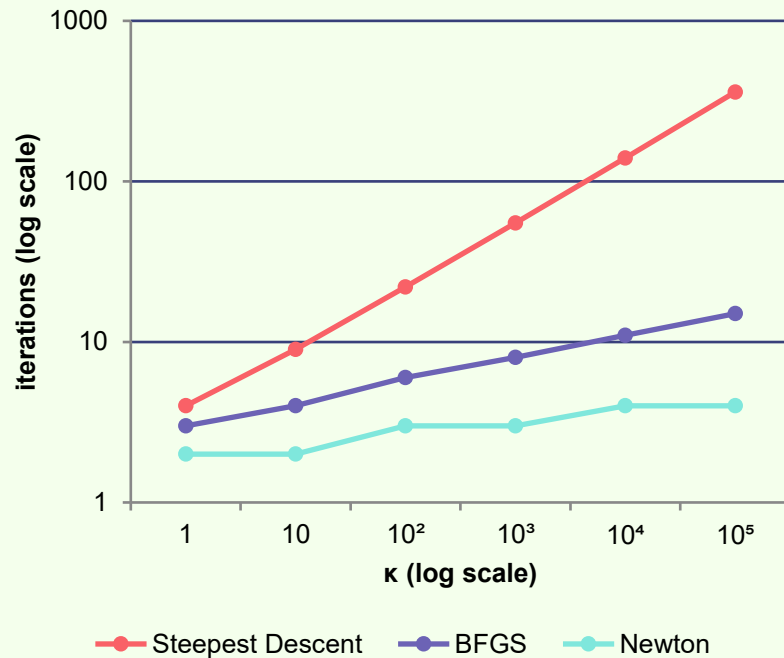
Runs on one fixed, stretched quadratic bowl — not whichever test function is selected in the dropdown.

Progressively increases κ (the ratio between the bowl's long and short axes) and records how many iterations each method needs to converge.

Only the step-size rule, tolerance, and max-iteration settings affect this chart — changing the picked function does not.

Kept isolated on purpose: if the function changed too, too many things would vary at once to draw a fair comparison.

Iterations vs. κ — Illustrative Trend



Steepest Descent's iteration count climbs steeply with κ ; Newton stays essentially flat; BFGS sits in between, exactly as the report describes.

Funkcija:

Himmelblau (4 minima)

Metoda:

Sve tri (poređenje)

Strategija koraka:

Egzaktna linijska pretraga

x0 (x):

4,0000

x0 (y):

4,0000

Maks. iteracija:

500

Tolerancija (eps):

1e-6

Animiraj korake

Pokreni

Studija uslovljenosti kappa

Resetuj

Strongly convex (quad + exp) | exact line search | x0=(2, -1.5) | eps=1e-06 | maxlter=500

steepest descent: Converged, 3 it, f*=0.995049, |g|=2.59343e-08, 0.04 ms

Newton: Converged, 2 it, f*=0.995049, |g|=1.38298e-08, 0.04 ms

BFGS: Converged, 2 it, f*=0.995049, |g|=4.30731e-07, 0.03 ms

Rosenbrock | exact line search | x0=(-1.2, 1) | eps=1e-06 | maxlter=500

steepest descent: MaxIterations, 500 it, f*=0.00129569, |g|=0.0421189, 3.49 ms

Newton: Converged, 13 it, f*=1.0141e-23, |g|=6.62179e-11, 0.12 ms

BFGS: Converged, 18 it, f*=4.62536e-20, |g|=9.47816e-09, 0.18 ms

kappa study (exact line search): kappa = 1 .. 1e4, eps = 1e-6

kappa=1e4: SD=74277, Newton=1, BFGS=2 iterations

Himmelblau (4 minima) | exact line search | x0=(0, 0) | eps=1e-06 | maxlter=500

steepest descent: Converged, 23 it, f*=2.68584e-15, |g|=4.22633e-07, 0.14 ms

Newton: Converged, 5 it, f*=8.36179e-20, |g|=3.3914e-09, 1 fallback(s), 0.06 ms

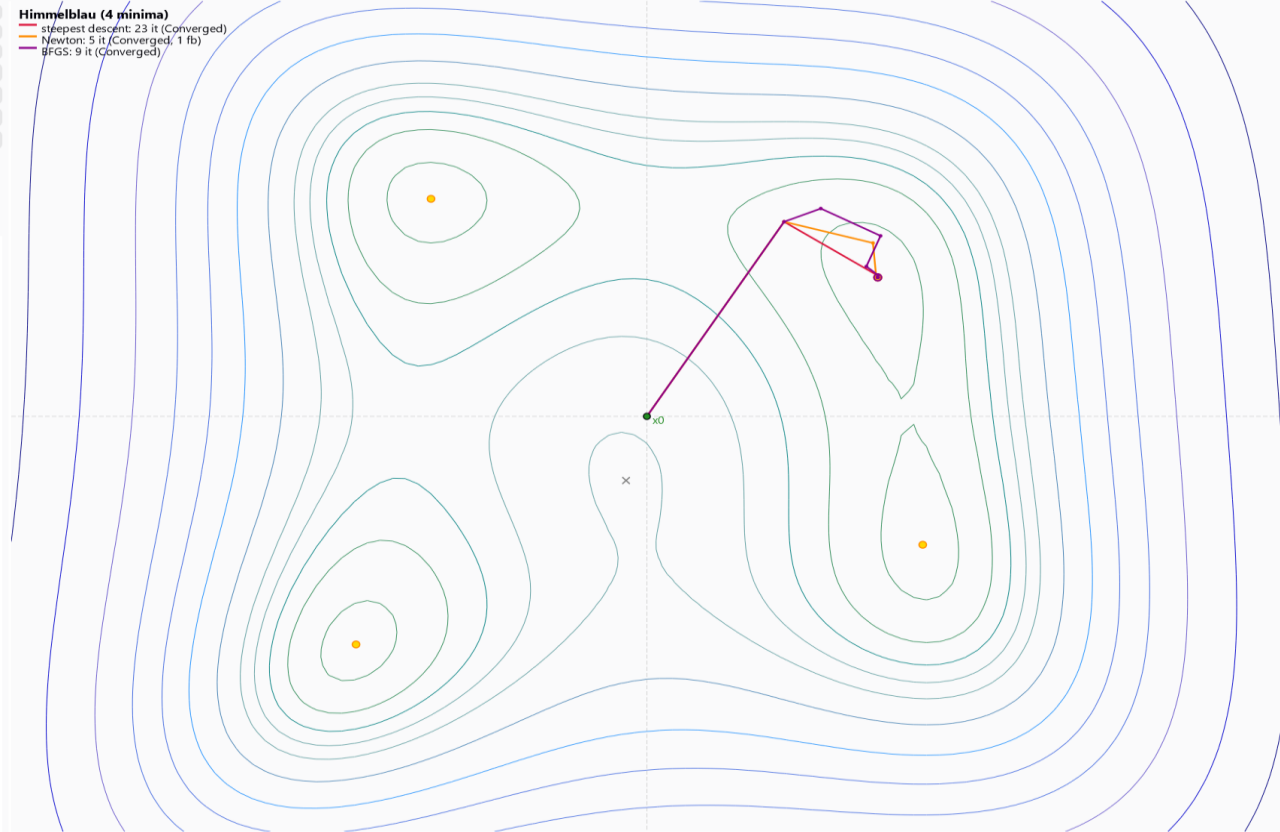
BFGS: Converged, 9 it, f*=9.77411e-25, |g|=1.0895e-11, 0.07 ms

kappa study (exact line search): kappa = 1 .. 1e4, eps = 1e-6

kappa=1e4: SD=74277, Newton=1, BFGS=2 iterations

kappa study (exact line search): kappa = 1 .. 1e4, eps = 1e-6

kappa=1e4: SD=74277, Newton=1, BFGS=2 iterations



Model / GUI Separation

MODEL LAYER

Objective.h

Math for each test function: value, gradient, Hessian

Optimizer.h

The shared loop all three methods run through

LineSearch.h

Exact line search / Armijo backtracking

CondStudy.h

Drives the condition-number hardness study

GUI LAYER

ControlsView.h

Control panel: method, function, run

ContourCanvas.h

Draws the contour plot and paths

ChartCanvas.h

Shared line-chart renderer

MainView.h

Assembles panel + charts into one screen